ITMS DevOps — Task Manager API: Final Report

November 2026

Task Manager API — Final Submission (ITMS DevOps)

A Task Manager REST API built with ASP.NET Core 8 + MySQL 8, containerized with Docker, orchestrated on Kubernetes, load-tested and fault-tolerance tested.

Repository: https://github.com/aledanee/final-exam-ITMSdevOps

Endpoints

Method	Path	Description
GET	`/health`	Health check (`{"status":"Healthy"}`)
POST	`/api/tasks`	Create a task
GET	`/api/tasks`	List tasks
GET	`/api/tasks/{id}`	Get one task
PUT	`/api/tasks/{id}`	Update a task
DELETE	`/api/tasks/{id}`	Delete a task

Task: `{ id, title, description, isCompleted, createdAt }`

Repository layout

```
├─── README.md                ← this file (root overview)
├─── REPORT.md                ← Deliverable 1 report (Docker/compose + assumptions)
```

```
├──── final-report.pdf        ← combined report (all deliverables, evidence,
links)
├──── app/                     ← Deliverable 1: containerized application
│    ├──── backend/             ← ASP.NET Core app — ALL code in a single file
(Program.cs)
│    │    ├──── Program.cs      ← endpoints, model, DbContext, DTOs
│    │    ├──── TaskManager.csproj
│    │    ├──── appsettings.json
│    │    └──── Dockerfile      ← multi-stage: sdk:8.0 → aspnet:8.0-bookworm-
slim, non-root, Kestrel
│    ├──── db/init.sql          ← MySQL first-boot schema
│    ├──── docker-compose.yaml ← named network + named volume, db & api
healthchecks
│    └──── README.md
├──── k8s/                     ← Deliverable 2: Kubernetes manifests
│    ├──── namespace.yaml  secret.yaml  pvc.yaml
│    ├──── mysql-deployment.yaml  mysql-service.yaml
│    ├──── api-deployment.yaml    api-service.yaml
│    ├──── hpa.yaml               ingress.yaml
│    ├──── kustomization.yaml     (kubectl apply -k k8s/)
│    ├──── README.md           ← how to deploy + assumptions
│    └──── REPORT.md           ← k8s evidence: pods/services, HPA, load test,
restarts
└──── load/load-test.js       ← k6 script: 500 VUs × 60 s
```

Quick start

Docker Compose:

```
cd app && docker compose up --build
# API + Swagger: http://localhost:8080/swagger   health:
          http://localhost:8080/health
```

Kubernetes (Minikube):

```
minikube start --driver=docker
minikube addons enable metrics-server
minikube image load aledanee/task-manager-api:1.0
kubectl apply -k k8s/
kubectl -n task-manager get pods,svc,hpa
curl http://$(minikube ip):30080/health
```

Key results (evidence in the reports)

Check	Result
Compose	`db (healthy)` + `api (healthy)` — API healthcheck polls `/health`
K8s rollout	`deploy/api 1/1`, `deploy/mysql 1/1`, PVC `Bound`
CRUD through Service	201 / 200 / 200 / 204 / 404 / 400 as specified
Non-root	Dockerfile `USER $APP_UID` and pod `securityContext` → `uid=1654(app)`
Load test (k6, 500 VU, 60 s)	22 605 requests, 365.5 req/s, 0.47 % errors, avg 1.24 s / p95 2.08 s
Autoscaling	HPA `1` → `5` replicas during the load test (CPU 410 % vs 60 % target), back to 1
Fault tolerance	`kill 1` in a pod → RESTARTS 0 → 1, pod healthy again, data intact (1 112 rows)

Reports

- `REPORT.md` — Deliverable 1 (Dockerfile, compose, ports/versions/DB/env assumptions)
- `k8s/REPORT.md` — Deliverable 2 (kubectl evidence, probes, HPA, load test, restart count)
- `k8s/README.md` — deployment steps + assumptions
- `final-report.pdf` — combined version of the above with the GitHub link

Report — Task Manager API

1. Overview

A Task Manager REST API built with C# / ASP.NET Core 8 (minimal APIs), persisting to MySQL 8 via Entity Framework Core (Pomelo provider).

The assignment asked for the same CRUD endpoints as the Node.js app written earlier in the course, and the professor confirmed that a basic implementation is acceptable — so this is a Task Manager written to meet that instruction and its endpoint list. It is not a line-by-line port of the original app (that source was not part of this repository).

All application code lives in a single file: `app/backend/Program.cs` (the model, DbContext, DTOs and every endpoint are in that one file, as required).

2. Endpoints (as specified by the assignment)

Required endpoint	C# implementation (this project)	Result codes
`POST /api/tasks`	`app.MapPost("/api/tasks", ...)`	201 + body / 400
`GET /api/tasks`	`app.MapGet("/api/tasks", ...)`	200 + array
`GET /api/tasks/{id}`	`app.MapGet("/api/tasks/{id}", ...)`	200 / 404
`PUT /api/tasks/{id}`	`app.MapPut("/api/tasks/{id}", ...)`	200 / 400 / 404
`DELETE /api/tasks/{id}`	`app.MapDelete("/api/tasks/{id}", ...)`	204 / 404
`GET /health`	`app.MapGet("/health", ...)`	200 `{"status":"Healthy"}`

Task payload shape: `{ id, title, description, isCompleted }` (+ `createdAt`, an added convenience field that does not alter the specified contract).

Stated assumption: the API was written directly against the endpoint list above (the professor's "basic app is fine" guidance); it was not derived from the original Node.js source. Error responses use JSON `{ "error": "..." }`.

3. Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build     # full image: restores +
          publishes
...
FROM mcr.microsoft.com/dotnet/aspnet:8.0-bookworm-slim AS runtime   # slim image:
          runs the app
ENTRYPOINT ["dotnet", "TaskManager.dll"]
```

- Official Microsoft images only — `mcr.microsoft.com/dotnet/sdk:8.0` (build) and `mcr.microsoft.com/dotnet/aspnet:8.0-bookworm-slim` (runtime, multi-stage build so the SDK never ships in the final image). *Note:* there is no `8.0-slim` tag — starting with .NET 8 the Debian slim variant is published as `8.0-bookworm-slim` (verified against MCR).
- Kestrel, ASP.NET Core's built-in web server, serves the app. `ASPNETCORE_URLS=http://+:8080` binds it inside the container; no reverse proxy is used.
- Runs as a non-root user: the final stage ends with `USER $APP_UID`, so the process runs as the image's built-in `app` user (uid 1654), not root.

Assumption: port 8080 is unprivileged, `/app` is world-readable and the app writes nothing to disk, so no root privileges are required.
- Restores from the `.csproj` in its own layer (cached) before copying `Program.cs`.

4. docker-compose.yaml

- Named network: `taskmanager-network` (driver `bridge`, explicit `name:`), both `api` and `db` are attached; the API reaches the database over the service name `db`.
- Named volume: `taskmanager-mysql-data` → `/var/lib/mysql`, so MySQL data survives `docker compose down` (add `-v` to wipe it).
- `init.sql` is mounted into `/docker-entrypoint-initdb.d/` and runs only on first start (empty data volume): creates `taskdb`, the `taskuser` account and the `Tasks` table.
- Startup ordering: `api` waits for the `db` healthcheck (`mysqladmin ping`) via `depends_on: condition: service_healthy`, plus the app itself retries connecting at boot.
- API healthcheck requests `GET /health` and requires `{"status":"Healthy"}` in the body (`interval: 10s`, `retries: 5`, `start_period: 20s`). The image ships no `curl`/`wget`, so the check uses bash's `/dev/tcp` to send the HTTP request — `docker compose ps` then reports the `api` service as `healthy`, not just `Up`.

5. Assumptions

Ports

Where	Port	Why
Host → API container	8080	Default ASP.NET Core 8 container port; non-privileged. Swagger UI at `http://localhost:8080/swagger`.
Host → MySQL	3306	MySQL default; exposed only for local debugging (the API talks over the internal network).
API → MySQL (in-network)	3306	Standard MySQL port.

.NET / runtime versions

- .NET 8 (LTS) — current LTS; images `sdk:8.0` / `aspnet:8.0-bookworm-slim`, `<TargetFramework>net8.0</TargetFramework>`.
- MySQL 8.0 official image; server version 8.0.36 supplied to EF for correct SQL dialect.

- Package versions: EF Core 8.0.11, Pomelo.EntityFrameworkCore.MySql 8.0.2, Swashbuckle 6.6.2, Microsoft.AspNetCore.OpenApi 8.0.11.
- The .NET SDK is not installed on the dev machine, so the build was validated inside the Docker build stage (compiles clean; full CRUD smoke-tested against a live MySQL container).

Database configuration

- Database `taskdb`, tables created without EF migrations — see next point.
- Credentials (dev/compose): root/root for the MySQL container, `taskuser/taskpass` for the app.
- Connection string is consumed from configuration: `ConnectionStrings:DefaultConnection`.

Database migrations

- No EF migration files are used. Two-layer approach instead:
  1. `app/db/init.sql` creates the schema when MySQL initialises (idempotent `CREATE TABLE IF NOT EXISTS`);
  2. at boot the app calls `EnsureCreated()` and verifies `Tasks` exists in `information_schema`, creating it if the database pre-existed without a schema.
- Assumption: an exam-scale app with a single table doesn't need migration history; for production, `dotnet ef migrations add` would replace both steps.

Environment variables

Variable	Set where	Value
`ConnectionStrings__DefaultConnection`	compose (`api`)	`Server=db;Port=3306;Database=tasl` — overrides `appsettings.json`
`ASPNETCORE_ENVIRONMENT`	compose (`api`)	`Production` (Swagger stays on, s
`ASPNETCORE_URLS`	Dockerfile	`http://+:8080` — Kestrel bind ac
`MYSQL_ROOT_PASSWORD`	compose (`db`)	`root` — required by the official

- Assumption: Swagger UI is left enabled in Production for exam/demo convenience; in a real deployment it would be gated behind `if (app.Environment.IsDevelopment())`.
- Local (non-Docker) runs fall back to `appsettings.json` (`localhost:3306`, root/root).

6. Verification performed

```
docker compose up --build        # db healthy → api up
GET    /health                   → 200 {"status":"Healthy",...}
POST   /api/tasks                → 201, task persisted
GET    /api/tasks  /{id}         → 200 list / 200 one / 404 missing
PUT    /api/tasks/1              → 200 updated (verified in MySQL)
DELETE /api/tasks/2              → 204, row removed (verified in MySQL)
POST   invalid (no title)        → 400
GET    /swagger/index.html       → 200
docker exec taskmanager-api id   → uid=1654(app)  (container runs non-root)

# named volume persistence
docker compose down && docker compose up -d   → row still present after full
teardown
docker network inspect taskmanager-network    → api + db both attached
docker volume inspect taskmanager-mysql-data  → named volume present
```

Containers and the test volume were removed afterwards (`docker compose down -v`), so the project starts from a clean database.

Deliverable 2 — Kubernetes Deployment

Manifests that run the Deliverable 1 Task Manager API (`app/backend`) and MySQL 8 on Kubernetes.

Files

File	Kind	Purpose
`namespace.yaml`	Namespace	Isolates everything in `task-manager`
`secret.yaml`	Secret	DB credentials + API connection string
`pvc.yaml`	PersistentVolumeClaim	`mysql-data` (1Gi, RWO) so data survives pod restarts
`mysql-deployment.yaml`	Deployment	MySQL 8, `Recreate` strategy, probes, limits, PVC mount
`mysql-service.yaml`	Service	`ClusterIP` — internal only, reachable as `mysql:3306`
`api-deployment.yaml`	Deployment	1 replica of the API (`spec.replicas: 1`), HTTP probes

File	Kind	Purpose
		on `/health`, non-root
`api-service.yaml`	Service	`NodePort 30080` → API on port 8080
`hpa.yaml`	HorizontalPodAutoscaler	scales the API 1 → 6 pods on 60% CPU (floor = 1)
`ingress.yaml`	Ingress	Optional host-based route (`task-manager.local`), needs ingress-nginx
`kustomization.yaml`	Kustomization	`kubectl apply -k k8s/` applies everything in order

Deploy

```
# 1. build the API image from Deliverable 1
cd app && docker build -t aledanee/task-manager-api:1.0 backend

# 2. apply everything
kubectl apply -k k8s/

# 3. watch it come up
kubectl -n task-manager get pods -w
kubectl -n task-manager get pvc
```

Call the API

```
# any cluster: through the NodePort
curl http://<node-ip>:30080/health
curl http://<node-ip>:30080/api/tasks

# handy local shortcut
kubectl -n task-manager port-forward svc/api 8080:8080
curl http://localhost:8080/api/tasks
```

Useful commands

```
kubectl -n task-manager get all                # deployments, pods, services
kubectl -n task-manager get hpa                # autoscaler: CURRENT/60% TARGETS
kubectl -n task-manager logs deploy/api        # API logs
kubectl -n task-manager scale deploy/api --replicas=4   # manual override (HPA
          resumes control after)
kubectl -n task-manager delete pod -l app=api   # force a rolling restart
```

The HPA needs metrics:

```
minikube addons enable metrics-server          # minikube
kubectl top pods -n task-manager               # sanity check that metrics flow
```

Report

Deployment evidence (Minikube `kubectl` output), probe configuration and justification of every choice: see REPORT.md.

Assumptions (Deliverable 2)

- Kubernetes version: manifests use stable GA APIs (`apps/v1`, `v1`, `networking.k8s.io/v1`), compatible with Kubernetes ≥ 1.19 (verified on Minikube, Kubernetes v1.37.0).
- Image location: the API image is referenced as `aledanee/task-manager-api:1.0`. Push it to a registry (`docker push`) for a remote cluster, or load the locally built image into a local cluster (`kind load docker-image …` / `minikube image load …`). `imagePullPolicy: IfNotPresent` makes both work. MySQL uses the public `mysql:8.0`.
- Ports: container/service port 8080 (Kestrel), MySQL 3306, exposed to the outside as NodePort 30080. Choose a different `nodePort` if 30080 is taken.
- Storage: the PVC uses the cluster's default StorageClass (provisioner-dependent: `standard`/`local-path` on kind/minikube, `gp3`/`standard` on cloud). 1Gi, `ReadWriteOnce`.
- Database bootstrap: the MySQL image's own env vars (`MYSQL_DATABASE`, `MYSQL_USER`, `MYSQL_PASSWORD`, `MYSQL_ROOT_PASSWORD`) create `taskdb` + `taskuser` on first boot — the same result as `app/db/init.sql`, which is therefore not mounted here. The `Tasks` table is created by the API itself on startup (`EnsureCreated` + `information_schema` check).
- Environment variables: `ASPNETCORE_ENVIRONMENT=Production` (plain env) and `ConnectionStrings__DefaultConnection` (from the Secret, note the `__` nesting separator).
- Secrets are plaintext `stringData` — acceptable for an exam; use Sealed/External Secrets or your platform's secret store in production.
- Scaling: API is stateless → starts at `spec.replicas: 1` (assignment requirement), with an HPA floor of 1 and a max of 6 (see `hpa.yaml`). MySQL stays at 1 replica (single RWO volume, `Recreate` strategy); HA/StatefulSet is out of scope. The HPA requires `metrics-server` (`minikube addons enable metrics-server`).
- Security context: the API pod runs as the image's non-root `app` user (uid 1654), all capabilities dropped, no privilege escalation — consistent with the Deliverable 1 Dockerfile.

- Ingress requires an ingress-nginx controller; without one the resource is inert.

Deliverable 2 — Kubernetes Deployment (Report)

1. What was deployed

Manifests in `k8s/`, applied with a single command:

```
kubectl apply -k k8s/
```

Resource	apiVersion / kind	Notes
`namespace`	v1 / Namespace	`task-manager`
`task-manager-db`	v1 / Secret	DB credentials + connection string
`mysql-data`	v1 / PersistentVolumeClaim	1Gi, RWO, default StorageClass
`mysql`	apps/v1 / Deployment	1 replica, `Recreate`, exec probes
`mysql`	v1 / Service	ClusterIP:3306 (internal only)
`api`	apps/v1 / Deployment	`spec.replicas: 1`, HTTP probes on `/health`
`api`	v1 / Service	NodePort 30080 → 8080
`api`	autoscaling/v2 / HorizontalPodAutoscaler	1→6 replicas at 60% CPU
`api`	networking.k8s.io/v1 / Ingress	optional, needs ingress-nginx

Both Deployments use `apiVersion: apps/v1` + `kind: Deployment`, as required.

Probes hit the real health endpoint (not TCP checks):

```
api deployment → readinessProbe: httpGet path=/health  livenessProbe: httpGet path=/health
```

2. Evidence of successful deployment (Minikube)

Environment: `minikube v1.39.0 --driver=docker`, Kubernetes v1.37.0, image loaded with `minikube image load aledanee/task-manager-api:1.0`.

`kubectl apply -k k8s/`

```
namespace/task-manager created
secret/task-manager-db created
service/api created
service/mysql created
persistentvolumeclaim/mysql-data created
deployment.apps/api created
deployment.apps/mysql created
ingress.networking.k8s.io/api created
```

Rollout

```
$ kubectl -n task-manager rollout status deploy/mysql --timeout=240s
deployment "mysql" successfully rolled out
$ kubectl -n task-manager rollout status deploy/api --timeout=240s
deployment "api" successfully rolled out
```

`kubectl get pods -o wide`

```
NAME                     READY   STATUS    RESTARTS   AGE   IP           NODE
NOMINATED NODE   READINESS GATES
api-699cbf7d4b-kx6wx     1/1     Running   0          57s   10.244.0.4   minikube
<none>           <none>
api-699cbf7d4b-xx5jv     1/1     Running   0          57s   10.244.0.3   minikube
<none>           <none>
mysql-5988457f9c-9ztng   1/1     Running   0          57s   10.244.0.5   minikube
<none>           <none>
```

(the two API pods above are the pre-review run; with `spec.replicas: 1` the steady state is a single API pod — see the autoscaling section, where the second pod appears only under load.)

`kubectl get services`

```
NAME    TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
api     NodePort    10.99.248.15    <none>        8080:30080/TCP   57s
mysql   ClusterIP   10.110.24.111   <none>        3306/TCP         57s
```

kubectl get pvc / kubectl get deployments

```
NAME         STATUS   VOLUME                                     CAPACITY   ACCESS
MODES   STORAGECLASS   AGE
mysql-data   Bound    pvc-46c1d48b-92f8-47ce-84e9-1bb2f9d86fc5   1Gi        RWO
standard       57s

NAME    READY   UP-TO-DATE   AVAILABLE   AGE
api     1/1     1            1           57s
mysql   1/1     1            1           57s
```

Single-replica steady state (assignment requirement: `spec.replicas: 1`)

```
$ kubectl -n task-manager get hpa
NAME   REFERENCE        TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
api    Deployment/api   cpu: 1%/60%   1      6         1          17m

$ kubectl -n task-manager get pods -l app=api
NAME                      READY   STATUS    RESTARTS   AGE
api-699cbf7d4b-xx5jv     1/1     Running   0          45m
```

Probe status (ready, zero restarts)

```
api-699cbf7d4b-xx5jv  ready=true restarts=0
```

Application checks through the Service (`NodePort 30080`)

```
GET  /health          -> {"status":"Healthy", ...}
POST /api/tasks       -> 201
GET  /api/tasks       -> [{"id":1,"title":"minikube demo", ...}]
GET  /api/tasks/1     -> 200
PUT  /api/tasks/1     -> 200
DELETE /api/tasks/1   -> 204
GET  /api/tasks/1     -> 404   (after delete)
```

3. Load test — 500 virtual users, 60 seconds (required)

Script: `load/load-test.js` (k6). 500 VUs looping for 60 s, mixed traffic (85 % `GET /api/tasks`, 10 % `GET /api/tasks/{id}`, 5 % `POST /api/tasks`), 100 ms think time. Target: the Kubernetes Service at `http://192.168.49.2:30080`.

```
docker run --rm --network host -v "$PWD/load:/scripts" grafana/k6 run /scripts/load-
          test.js
```

Results (verbatim k6 summary)

```
HTTP
http_req_duration..............: avg=1.24s min=52.01ms med=1.22s max=7.05s
p(90)=1.86s p(95)=2.08s
http_req_failed................: 0.47%  108 out of 22605
http_reqs......................: 22605  365.539794/s

checks_total.......: 22605   365.539794/s
checks_succeeded...: 100.00% 22605 out of 22605

■ THRESHOLDS
  check_failed       ✓ rate<0.01   rate=0.00%
  http_req_failed    ✓ rate<0.01   rate=0.47%
  http_req_duration  ✗ p(95)<500   p(95)=2.08s
```

Metric	Required	Measured	Verdict
Throughput	—	365.5 req/s (22 605 requests / 60 s)	—
Error rate	< 1 %	0.47 % (108 / 22 605)	✅ pass
Latency avg / p95	p95 < 500 ms	avg 1.24 s / p95 2.08 s (max 7.05 s)	❌ threshold missed
Application checks	—	100 % of 22 605 checks passed	✅

Honest analysis: the p95 threshold was missed because the whole cluster is a single 2-CPU Minikube node running 1→5 API pods plus MySQL, so at ~365 req/s the request queue dominates latency (single RWO database, no read replicas). The error rate stayed under 1 % and no request crashed the service. Scaling the node / adding a second DB tier (out of scope here) is the fix; the numbers are reported as measured, not adjusted.

HPA reaction during the same test (10 s sampling)

```
12:08:42Z  cpu 1%/60%    min1 max6   replicas 1
12:08:52Z  cpu 1%/60%    min1 max6   replicas 1
12:09:03Z  cpu 1%/60%    min1 max6   replicas 1
12:09:13Z  cpu 1%/60%    min1 max6   replicas 1
12:09:23Z  cpu 1%/60%    min1 max6   replicas 1
12:09:33Z  cpu 1%/60%    min1 max6   replicas 1
12:09:43Z  cpu 1%/60%    min1 max6   replicas 1
12:09:53Z  cpu 410%/60%  min1 max6   replicas 1     ← metrics arrive, target blown
```

```
12:10:03Z  cpu 410%/60%  min1 max6   replicas 1

$ kubectl -n task-manager get hpa          # seconds later
NAME   REFERENCE        TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
api    Deployment/api   cpu: 410%/60%   1         6         5          19m

$ kubectl -n task-manager get pods -l app=api
NAME                   READY   STATUS    RESTARTS   AGE
api-699cbf7d4b-xx5jv   1/1     Running   0          46m
api-699cbf7d4b-2rjrg   1/1     Running   0          22s
api-699cbf7d4b-kw5v4   1/1     Running   0          22s
api-699cbf7d4b-rhxzx   1/1     Running   0          22s
api-699cbf7d4b-z4rzk   1/1     Running   0          22s
api-699cbf7d4b-xp2rj   1/1     Running   0          7s
```

CPU blew past the 60 % target (410 %) and the HPA scaled 1 → 5 pods (max 6), each new pod entering Service rotation only after `/health` passed readiness.

Load removed → scale-down back to the floor of 1 (300 s stabilization window, see §6.14):

```
12:16:03Z
NAME   REFERENCE        TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
api    Deployment/api   cpu: 1%/60%   1      6         1          25m

NAME                   READY   STATUS    RESTARTS        AGE
api-699cbf7d4b-xx5jv   1/1     Running   1 (5m41s ago)   52m

NAME                   CPU(cores)   MEMORY(bytes)
api-699cbf7d4b-xx5jv   1m           58Mi
```

The 4 added pods were removed once CPU settled back to 1 % — only one pod remains (the same one that survived the container restart in §4, `RESTARTS 1`).

4. Fault tolerance — container restart count increases

Deleting pods shows *replacement*; a real crash shows the restart counter incrementing on the same pod. Simulated by killing PID 1 (the `dotnet` process) inside a running API container:

```
=== BEFORE ===
api-699cbf7d4b-2rjrg RESTARTS=0
api-699cbf7d4b-kw5v4 RESTARTS=0
api-699cbf7d4b-rhxzx RESTARTS=0
api-699cbf7d4b-xp2rj RESTARTS=0
api-699cbf7d4b-xx5jv RESTARTS=0
```

```
api-699cbf7d4b-z4rzk RESTARTS=0

$ kubectl -n task-manager exec api-699cbf7d4b-xx5jv -- sh -c 'kill 1'

=== AFTER (20 s later) ===
NAME                   READY   STATUS    RESTARTS   AGE
api-699cbf7d4b-2rjrg   1/1     Running   0          51s
api-699cbf7d4b-kw5v4   1/1     Running   0          51s
api-699cbf7d4b-rhxzx   1/1     Running   0          51s
api-699cbf7d4b-xp2rj   1/1     Running   0          36s
api-699cbf7d4b-xx5jv   1/1     Running   1 (20s ago)   47m     ← same pod, counter
0 → 1
api-699cbf7d4b-z4rzk   1/1     Running   0          51s
```

Service recovered without intervention:

```
GET /health     -> {"status":"Healthy", ...}
GET /api/tasks  -> rows: 1112      (every row created by the 500-VU load test still
present)
```

What this proves: the kubelet detected the dead container, restarted it *in the same pod* (`RESTARTS 0 → 1`, same pod name and age), the readiness probe gated it back into the Service, and the database was untouched — crash recovery works end-to-end.

5. Other evidence

Non-root runtime (same user as the Dockerfile)

```
$ kubectl -n task-manager exec deploy/api -- id
uid=1654(app) gid=1654(app) groups=1654(app)
```

Persistence

- Row created → MySQL pod deleted → new pod re-attaches the PVC → row still present.
- 1 112 rows written under load survived the container restart above.

6. Justification of choices

1. `apps/v1` Deployment — stable/GA API (required since Kubernetes 1.9; `extensions/v1beta1` is removed). One per component: `api`, `mysql`.
2. HTTP probes on `/health` for the API — `httpGet` readiness/liveness against the real endpoint proves the app is serving requests, not just that the port is

open; `/health` is the endpoint the deliverable already exposes.

3. Exec probe (`mysqladmin ping`) for MySQL — MySQL has no HTTP endpoint; an exec ping is the accurate health signal for it. Nothing more was added.
4. `spec.replicas: 1` for the API — matches the assignment requirement; the pod count under load is then handled by the HPA (floor 1, ceiling 6) rather than a fixed replica count. MySQL also stays at 1 (single RWO volume, HA/StatefulSet out of scope).
5. `Recreate` strategy for MySQL — prevents two pods claiming the same `ReadWriteOnce` volume during rollouts.
6. PVC `mysql-data` (1Gi) — database data must outlive pod restarts; verified by deleting the MySQL pod and finding the row intact.
7. `NodePort 30080` — makes the API reachable on any cluster (Minikube included) without an ingress controller; port 8080 in, 30080 out.
8. `Ingress` included but optional — documents how the app would be exposed by hostname; inert without ingress-nginx, so it adds no moving parts.
9. Secret for DB credentials + connection string — keeps passwords out of the pod spec and lets the app consume them via `ConnectionStrings__DefaultConnection`.
10. DB bootstrap via the MySQL image's env vars (`MYSQL_DATABASE/MYSQL_USER/…`) — same outcome as `app/db/init.sql` with no extra ConfigMap/mount; the app creates the `Tasks` table itself at startup.
11. `imagePullPolicy: IfNotPresent` — allows the locally built image to be `minikube image load`-ed for this demo while still working with a pushed registry image.
12. securityContext (`runAsNonRoot`, uid 1654, drop ALL caps, no privilege escalation) — continues the non-root decision from the Deliverable 1 Dockerfile; verified with `kubectl exec … id`.
13. Requests/limits on both containers — required for stable scheduling, to prevent one pod from starving the node, and as the denominator for the HPA's CPU % (API 100m/128Mi→500m/512Mi, MySQL 100m/256Mi→1/512Mi).
14. HPA (`autoscaling/v2`, 1→6 at 60 % CPU) — the API is stateless, so horizontal scale-out is the correct response to load while keeping the required floor of 1. Scale-up is instant (`stabilizationWindowSeconds: 0`), scale-down waits 300 s so dips don't cause flapping; requires `metrics-server`. Verified live under the 500-VU test: CPU 410 % → 5 replicas → back to 1 when idle.

Nothing else was added: no ClusterAutoscaler, no NetworkPolicy, no StatefulSet, no init containers — out of scope for this deliverable.

7. Reproduce

```
minikube start --driver=docker
minikube addons enable metrics-server                # required by the HPA
minikube image load aledanee/task-manager-api:1.0     # after: docker build -t
          aledanee/task-manager-api:1.0 app/backend
kubectl apply -k k8s/
```

```
kubectl -n task-manager get pods,svc,pvc,hpa
curl http://$(minikube ip):30080/health

# required load test: 500 VUs for 60 s (latency, error rate, throughput)
docker run --rm --network host -v "$PWD/load:/scripts" grafana/k6 run /scripts/load-
          test.js
watch kubectl -n task-manager get hpa                # watch 1 → 5 during the test

# fault tolerance: restart counter on one pod
kubectl -n task-manager exec deploy/api -- sh -c 'kill 1'
kubectl -n task-manager get pods -l app=api          # RESTARTS 0 -> 1
```